CSE 2006 - Programming in Java

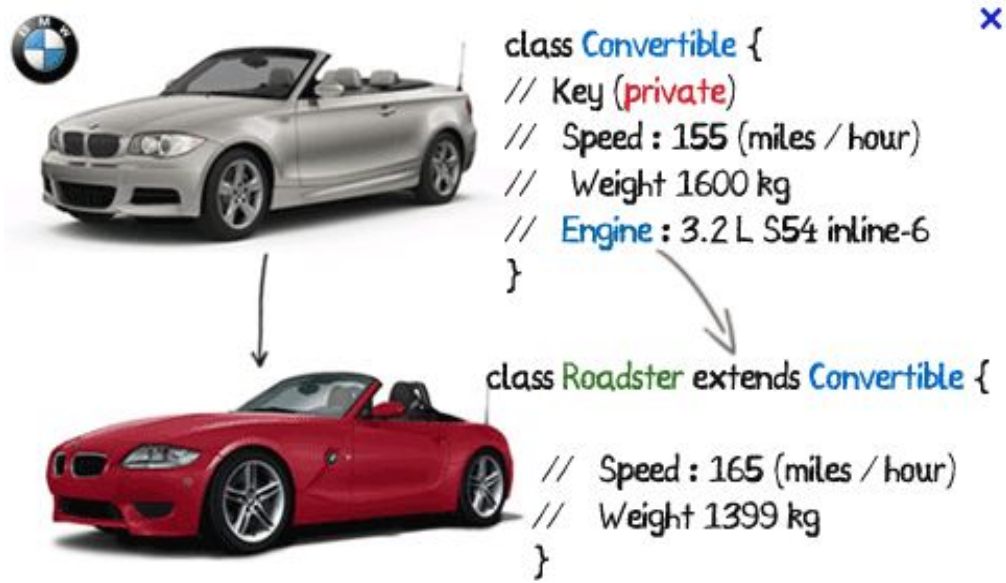
Course Type: LP Credits: 3

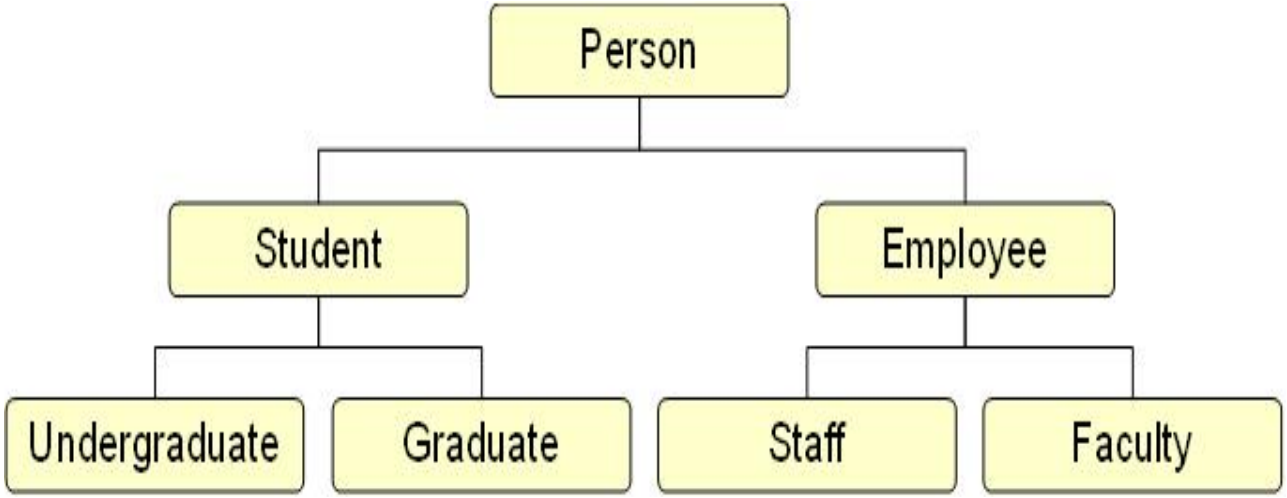
Prepared by
C.S Rajpoot
School of Computing Science and Engineering
VIT Bhopal University

Unit-2 Java Object-oriented Programming

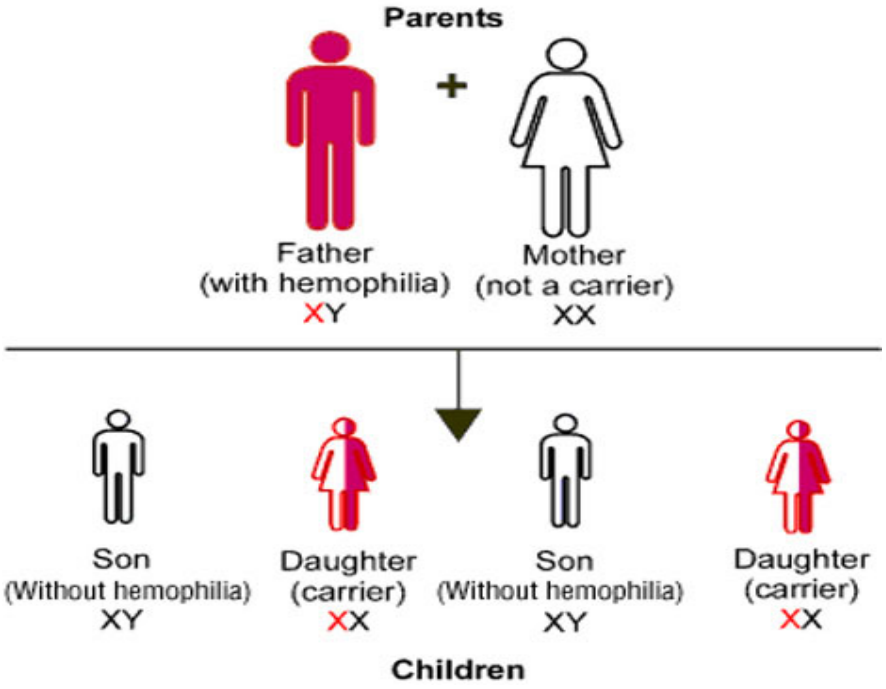
- **Java OOP (Basics)**
- Java Class and Objects, Java Methods, Java Constructor, Java Strings, Java Access Modifiers, Java this keyword, Java final keyword, Java Recursion, Java instance of Operator, Java Single Class and Anonymous Class, Java enum Class
- **Java OOP (Inheritance & Polymorphism)**
- Java Inheritance, Java Method Overriding, Java super Keyword, Abstract Class & Method, Java Interfaces, Java Polymorphism (overloading & overriding), Java Encapsulation
- **Java OOP (Other types of classes)**
- Nested & Inner Class, Java Static Class, Java Anonymous Class, Java Singleton, Java enum Class, Java enum Constructor, Java enum String, Java Reflection

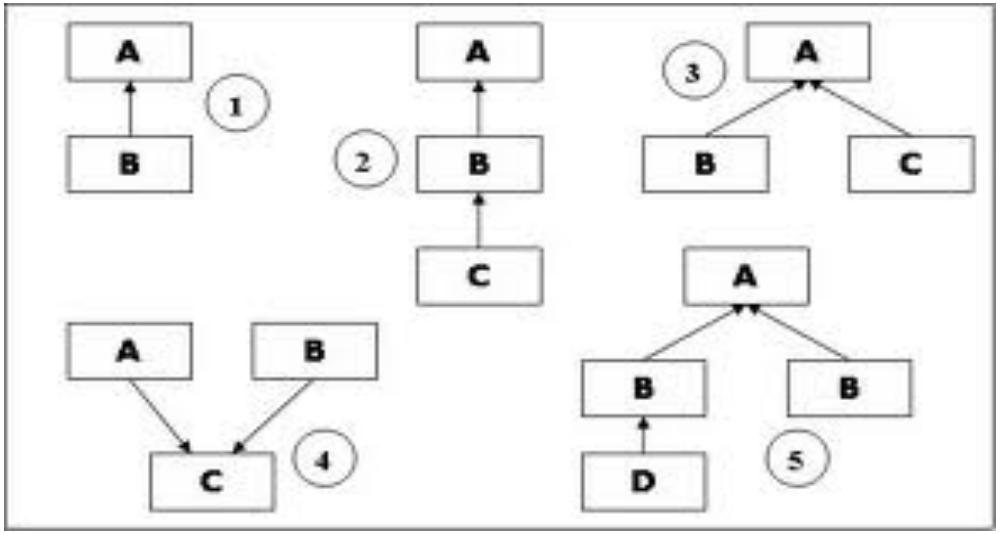
What do you understand?





What do you understand?





Inheritance Basics

- Inheritance is the process by which one object acquires the properties of another objects
- By use of inheritance an object need to define only qualities that make it unique within class
- It can inherit its general attributed from its parent
- It provides code **re-usability** feature
- Only single inheritance is allowed among classes
- All public and protected members of a super class are accessible in the subclasses
- All protected members are also accessible within the package

Inheritance Basics

```
// A simple example of inheritance.

// Create a superclass.
class A {
  int i, j;

  void showij() {
    System.out.println("i and j: " + i + " " + j);
  }
}

// Create a subclass by extending class A.
class B extends A {
  int k;

  void showk() {
    System.out.println("k: " + k);
  }
  void sum() {
    System.out.println("i+j+k: " + (i+j+k));
  }
}
```

Inheritance Basics

```
class SimpleInheritance {
  public static void main(String args[]) {
    A superOb = new A();
    B subOb = new B();

    // The superclass may be used by itself.
    superOb.i = 10;
    superOb.j = 20;
    System.out.println("Contents of superOb: ");
    superOb.showij();
    System.out.println();

    /* The subclass has access to all public members of
       its superclass. */
    subOb.i = 7;
    subOb.j = 8;
    subOb.k = 9;
    System.out.println("Contents of subOb: ");
    subOb.showij();
    subOb.showk();
    System.out.println();

    System.out.println("Sum of i, j and k in subOb:");
    subOb.sum();
  }
}
```

output:

Contents of superOb:
i and j: 10 20
Contents of subOb:
i and j: 7 8
k: 9
Sum of i, j and k in subOb:
i+j+k: 24

Member Access and Inheritance

```
/* In a class hierarchy, private members remain private to their class.
This program contains an error and will not compile. */
// Create a superclass.
class A {
int i; // public by default
private int j; // private to A
void setij(int x, int y) {
i = x;
j = y;
} }
// A's j is not accessible here.
class B extends A {
int total;
void sum() {
total = i + j; // ERROR, j is not accessible here
}
}
```

```
class Access {
public static void main(String args[]) {
B subOb = new B();
subOb.setij(10, 12);
subOb.sum();
System.out.println("Total is " + subOb.total);
}
}
```

A class member that has been declared as private will remain private to its class. It is not accessible by any code outside its class, including subclasses.

What are the types of inheritance?

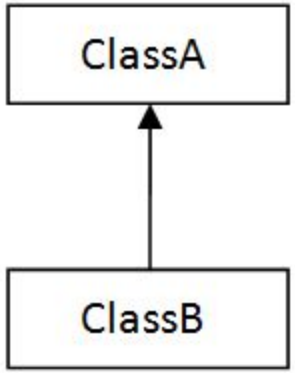
Single inheritance: One super class and single sub class.

Multiple inheritance: More than one super class and single subclass.

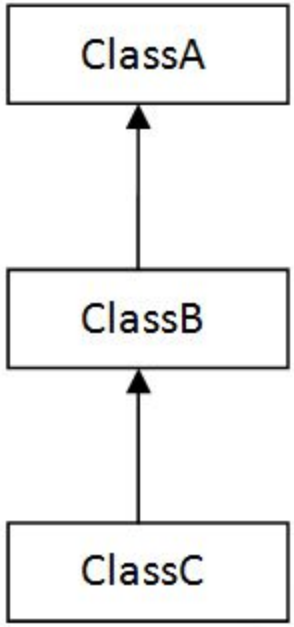
Hierarchial inheritance: one super class and more than one subclass.

Multilevel inheritance : Deriving a sub class from another sub class

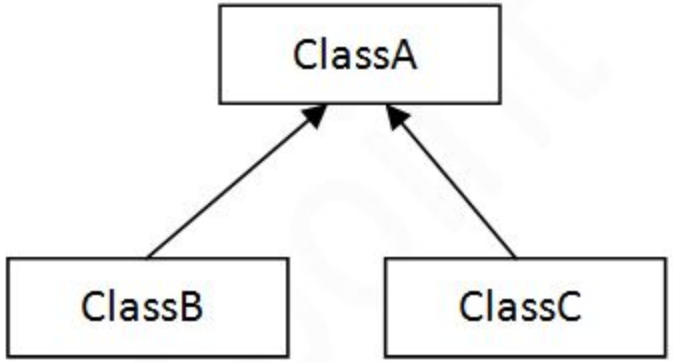
Types of inheritance in java



1) Single

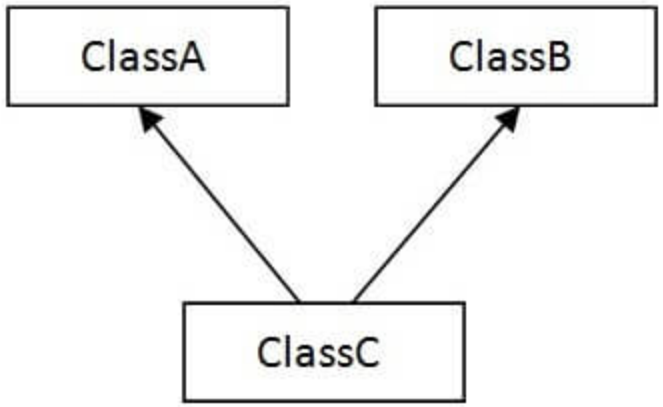


2) Multilevel

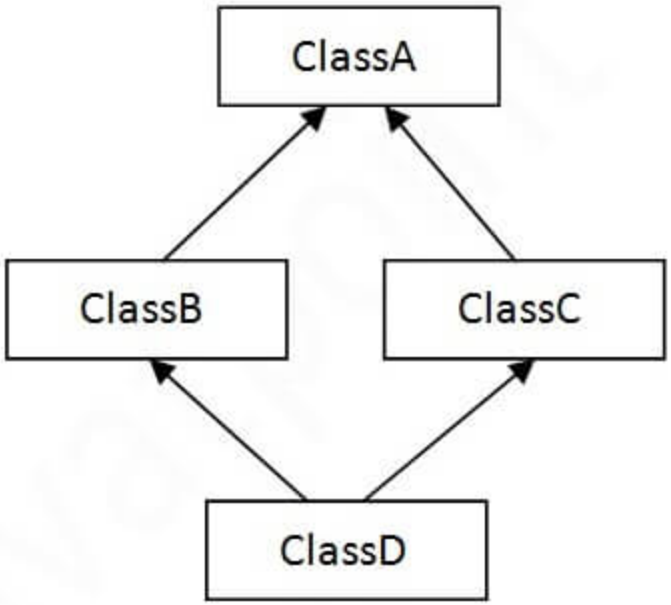


3) Hierarchical

Types of inheritance in java



4) Multiple



5) Hybrid

Single Inheritance Example

When a class inherits another class, it is known as a *single inheritance*. In the example given below, Dog class inherits the Animal class, so there is the single inheritance.

File: TestInheritance.java

```
class Animal{
void eat()
{System.out.println("eating...");}
}
class Dog extends Animal{
void bark()
{System.out.println("barking...");}
}
class TestInheritance{
public static void main(String args[]){
Dog d=new Dog();
d.bark();
d.eat();
}}
```

Output:
barking...
eating…

Multilevel Inheritance Example

When there is a chain of inheritance, it is known as *multilevel inheritance*. As you can see in the example given below, BabyDog class inherits the Dog class which again inherits the Animal class, so there is a multilevel inheritance.

File: TestInheritance2.java

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class BabyDog extends Dog{
void weep(){System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}}
```

Output:

```
weeping... barking...
eating...
```

Hierarchical Inheritance Example

When two or more classes inherits a single class, it is known as hierarchical inheritance. In the example given below, Dog and Cat classes inherits the Animal class, so there is hierarchical inheritance.

File: TestInheritance3.java

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void bark(){System.out.println("barking...");}
}
class Cat extends Animal{
void meow(){System.out.println("meowing...");}
}
class TestInheritance3{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
c.eat();
//c.bark();        //C.T.Error
}}
```

Output:

meowing... eating...

Hierarchical Inheritance Example

Q) Why multiple inheritance is not supported in java?

To reduce the complexity and simplify the language, multiple inheritance is not supported in java.

Consider a scenario where A, B, and C are three classes. The C class inherits A and B classes. If A and B classes have the same method and you call it from child class object, there will be ambiguity to call the method of A or B class.

Since compile-time errors are better than runtime errors, Java renders compile-time error if you inherit 2 classes. So whether you have same method or different, there will be compile time error.

```
class A{
void msg(){System.out.println("Hello");}  }
class B{
void msg(){System.out.println("Welcome");}
}
class C extends A,B{      //suppose if it were
public static void main(String args[]){
  C obj=new C();
  obj.msg();//Now which msg() method would be invoked?
} }
```

Compile Time Error

Access Modifiers in Java

There are four types of Java access modifiers:

Private: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

Default: The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.

Protected: The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.

Public: The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

There are many non-access modifiers, such as static, abstract, synchronized, native, volatile, transient, etc. Here, we are going to learn the access modifiers only.

Access Modifiers in Java

Understanding Java Access Modifiers

Let's understand the access modifiers in Java by a simple table.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

https://www.javatpoint.com/access-modifiers

Super Keyword in Java

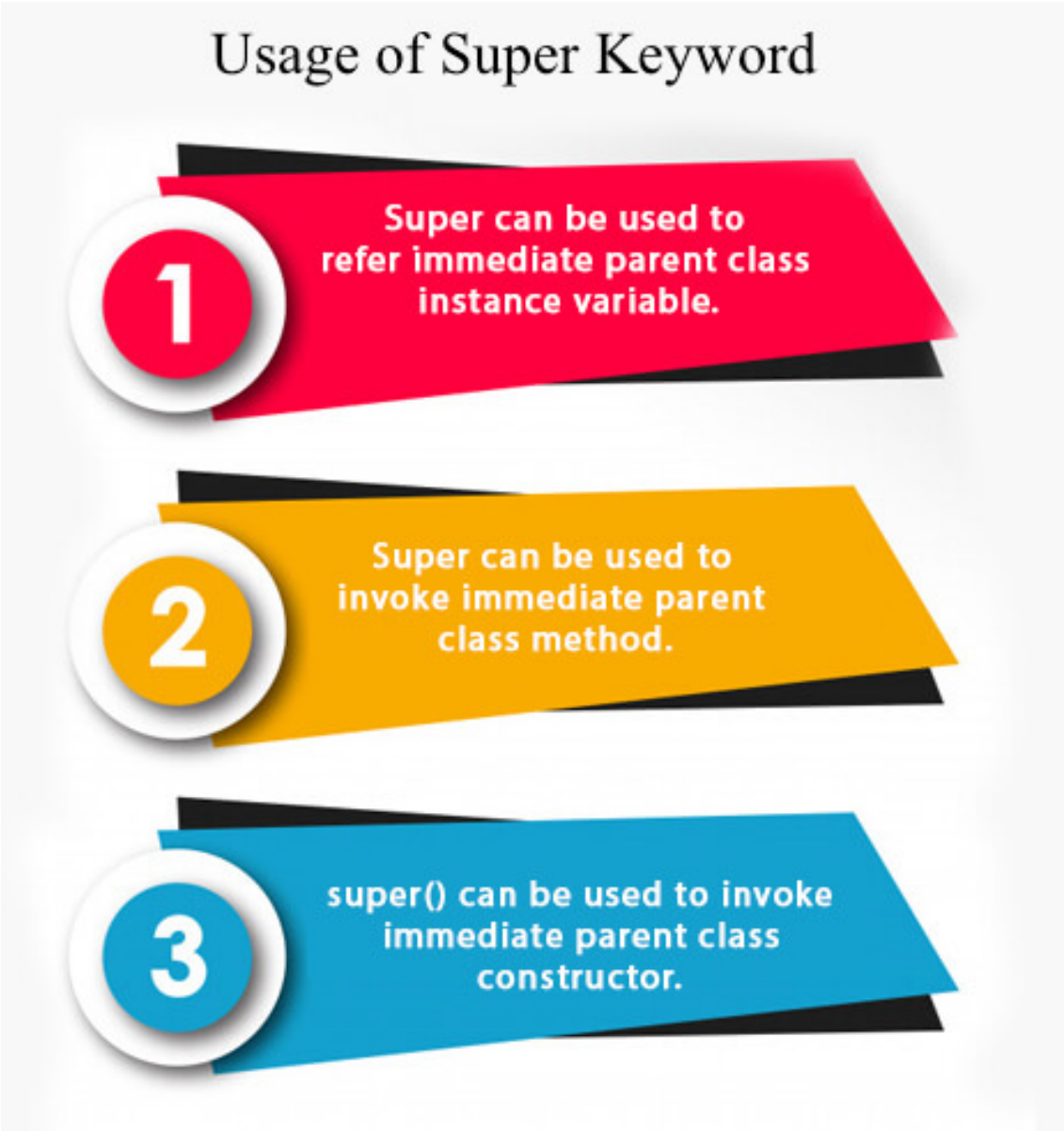
- The super keyword in Java is a reference variable which is used to **refer immediate parent class object**
- Whenever you create the instance of subclass, an instance of parent class is created implicitly which is referred by **super reference variable**

Usage of Java super Keyword

- super can be used to refer immediate parent class instance variable
- super can be used to invoke immediate **parent class method**
- super() can be used to invoke immediate parent class constructor

https://www.javatpoint.com/super-keyword

Super Keyword in Java



black white

Super Keyword in Java

1) super is used to refer immediate parent class instance variable.

- We can use super keyword to access the data member or field of parent class
- It is used if parent class and child class have same fields

```
class Animal{
String color="white";
}
class Dog extends Animal{
String color="black";
void printColor(){
System.out.println(color);        //prints color of Dog class
System.out.println(super.color);     //prints color of Animal class
}
}
class TestSuper1{
public static void main(String args[]){
Dog d=new Dog();
d.printColor();
}}
```

Output:
black
white

black white

Super Keyword in Java

- In the previous slide, Animal and Dog both classes have a common property color.
- If we print color property, it will print the color of current class by default.
- To access the parent property, we need to use super keyword.

black white

Super Keyword in Java

2) super can be used to invoke parent class method

The super keyword can also be used to invoke parent class method. It should be used if subclass contains the same method as parent class. In other words, it is used if method is overridden.

```
class Animal{
void eat(){System.out.println("eating...");}
}
class Dog extends Animal{
void eat(){System.out.println("eating bread...");}
void bark()  {System.out.println("barking...");}
void work(){
super.eat();  //invoke parent class method
bark();
}
}
class TestSuper2{
public static void main(String args[]){
Dog d=new Dog();
d.work();
}}
```

Output:
eating...
barking...

black white

Super Keyword in Java

- In the above example Animal and Dog both classes have eat() method if we call eat() method from Dog class, it will call the eat() method of Dog class by default because priority is given to local.
- **To call the parent class method, we need to use super keyword.**

black white

Super Keyword in Java

3) super is used to invoke parent class constructor.

The super keyword can also be used to invoke the parent class constructor. Let's see a simple example:

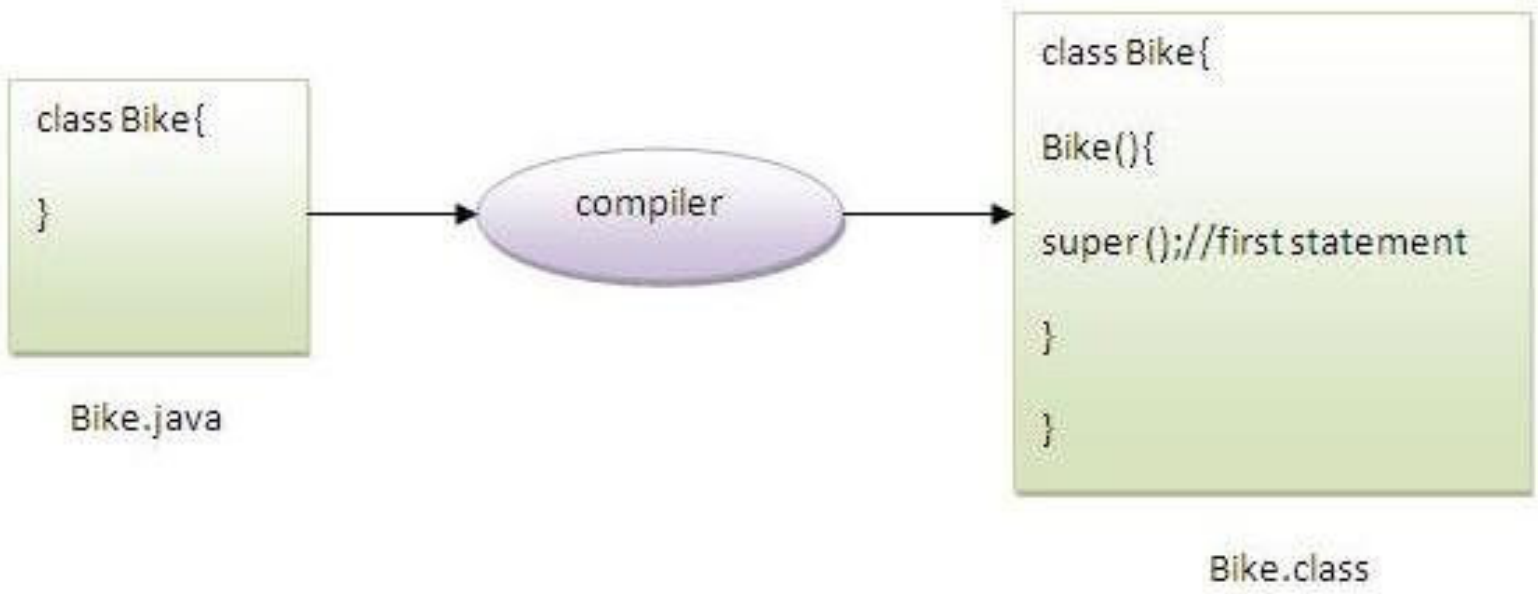
```
class Animal{
Animal(){System.out.println("animal is created");}
}
class Dog extends Animal{
Dog(){
super();
System.out.println("dog is created");
}
}
class TestSuper3{
public static void main(String args[]){
Dog d=new Dog();
}}
```

Output:
animal is created
dog is created

Super Keyword in Java

Note: **super() is added in each class constructor automatically by compiler if there is no super() or this().**

As we know well that default constructor is provided by compiler automatically if there is no constructor.
But, it also adds super() as the first statement.



Super Keyword in Java

Another example of super keyword where super() is provided by the compiler implicitly.

```
class Animal{
Animal(){System.out.println("animal is created");}
}
class Dog extends Animal{
Dog(){
System.out.println("dog is created");
}
}
class TestSuper4{
public static void main(String args[]){
Dog d=new Dog();
}}
```

Output:
animal is created
dog is created

Super Keyword in Java

super example: real use

Here, Emp class inherits Person class so all the properties of Person will be inherited to Emp by default. To initialize all the property, we are using parent class constructor from child class. In such way, we **are reusing the parent class constructor.**

```
class Person{
int id;
String name;
Person(int id,String name){
this.id=id;
this.name=name;
} }
class Emp extends Person{
float salary;
Emp(int id,String name,float salary){
super(id,name);   //reusing parent constructor
this.salary=salary;  }
void display(){System.out.println(id+" "+name+" "+salary);}
}
```

```
class TestSuper5{
public static void main(String[] args)
{
Emp e1=new Emp(1,"ankit",45000f);
e1.display();
}}
```

Output:
1 ankit 45000

Method Overriding

- In a class hierarchy, when a method in a subclass has the **same name and type signature** as a method in its superclass
- Then the method in the subclass is said to **override** the method in the Superclass
- When an overridden method is called from within a subclass, it will always refer to the version of that method defined by the subclass

Usage of Java Method Overriding

- Method overriding is used to provide the specific implementation of a method which is **already provided by its superclass**
- **Method overriding is used for runtime polymorphism**

Rules for Java Method Overriding

- The method must have the **same name as in the parent class**
- The method must have the **same parameter as in the parent class**
- There must be an **IS-A relationship (inheritance)**

Method Overriding

- Reference:
- **Method Overloading and Overriding**

https://youtu.be/KqAXwUcMR9M

https://youtu.be/jg4MpYr1TBc

https://www.javatpoint.com/method-overriding-in-java

Method Overriding

```
class A {
int i, j;
A(int a, int b) {
i = a;
j = b;
}
// display i and j
void show() {
System.out.println("i and j: " + i + " " + j);
}
}
class B extends A {
int k;
B(int a, int b, int c) {
super(a, b);
k = c;
}
// display k – this overrides show() in A
void show() {
System.out.println("k: " + k);
}
}
```

```
class Override {
public static void main(String args[]) {
B subOb = new B(1, 2, 3);
subOb.show(); // this calls show() in B
}
}
```

Output:

k: 3

// Methods with differing type signatures are overloaded – not overridden.

```
class A {
int i, j;
A(int a, int b) {
i = a;
j = b;
}
// display i and j
void show() {
System.out.println("i and j: " + i + " " + j);
}}
// Create a subclass by extending class A.
class B extends A {
int k;
B(int a, int b, int c) {
super(a, b);
k = c;
}
// overload show()
void show(String msg) {
System.out.println(msg + k);
}}
```

```
class Override {
public static void main(String args[]) {
B subOb = new B(1, 2, 3);
subOb.show("This is k: ");
                // this calls show() in B
subOb.show(); // this calls show() in A
}
}
```

Output:

This is k: 3
i and j: 1 2

Understanding the problem without method overriding

Let's understand the problem that we may face in the program if we don't use method overriding.

```
//Java Program to demonstrate why we need method overriding
//Here, we are calling the method of parent class with child

//Creating a parent class
class Vehicle
{   void run()  {System.out.println("Vehicle is running");   }
}
//Creating a child class
class Bike extends Vehicle{
  public static void main(String args[]){
  //creating an instance of child class
  Bike obj = new Bike();
  //calling the method with child class instance
  obj.run();
  }
}
```

Output:
Vehicle is running

Example of method overriding

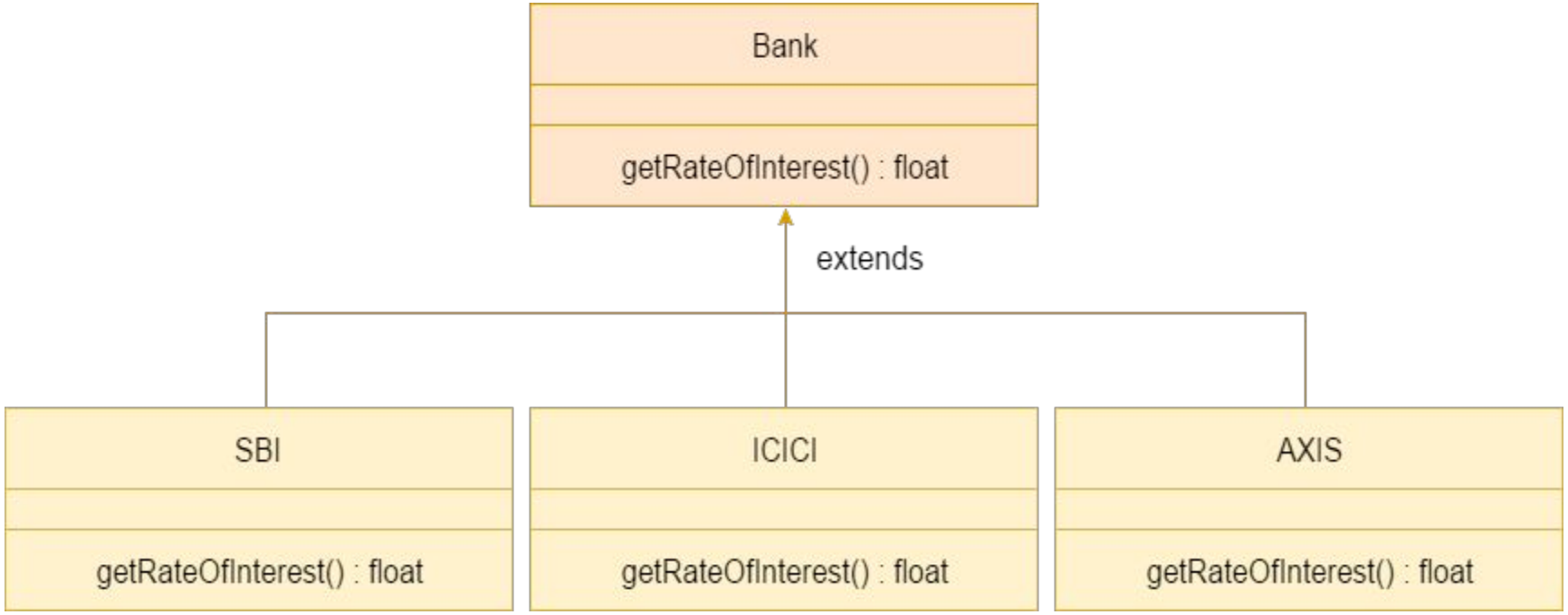
In this example, we have defined the run method in the subclass as defined in the parent class but it has some specific implementation. The name and parameter of the method are the same, and there is IS-A relationship between the classes, so there is method overriding.

```
//Java Program to illustrate the use of Java Method Overriding
//Creating a parent class.
class Vehicle{
  //defining a method
  void run(){System.out.println("Vehicle is running");}  }
//Creating a child class
class Bike2 extends Vehicle{
  //defining the same method as in the parent class
  void run(){System.out.println("Bike is running safely");}
  public static void main(String args[]){
  Bike2 obj = new Bike2(); //creating object
  obj.run(); //calling sub class method
  }
}
```

Output:
Bike is running safely

A real example of Java Method Overriding

Consider a scenario where Bank is a class that provides functionality to get the rate of interest. However, the rate of interest varies according to banks. For example, SBI, ICICI and AXIS banks could provide 8%, 7%, and 9% rate of interest.



//Java Program to demonstrate the real scenario of Java Method Overriding

```
//Creating a parent class.
class Bank{
int getRateOfInterest(){return 0;}
}
//Creating child classes.
class SBI extends Bank
{  int getRateOfInterest(){return 8;}  }
class ICICI extends Bank
{  int getRateOfInterest(){return 7;}  }
class AXIS extends Bank
{  int getRateOfInterest(){return 9;}  }
//Test class to create objects and call the methods
class Test2
{
public static void main(String args[]){
SBI s=new SBI();
ICICI i=new ICICI();
AXIS a=new AXIS();
System.out.println("SBI Rate of Interest: "+s.getRateOfInterest());
System.out.println("ICICI Rate of Interest: "+i.getRateOfInterest());
System.out.println("AXIS Rate of Interest: "+a.getRateOfInterest());
}}
```

Output:

SBI Rate of Interest: 8
ICICI Rate of Interest: 7
AXIS Rate of Interest: 9

Polymorphism in Java

- **Polymorphism in Java** is a concept by which we can perform a ***single action in different ways***.
- Polymorphism is derived **from 2 Greek words: poly and morphs**.
- The word "poly" means many and "morphs" means forms. So polymorphism means **many forms.**
- There are two types of polymorphism in Java:
  1. **compile-time polymorphism**
  2. **runtime polymorphism.**
- We can perform polymorphism in java by method **overloading and method overriding.**
- If you overload a static method in Java, it is the example of **compile time polymorphism.**

Why Overridden Methods?

- Overridden methods allow Java to support **run-time polymorphism.**
- Polymorphism is essential to object-oriented programming for **one reason:**
  - it allows a general class to specify **methods that will be common to all of its derivatives,** while allowing subclasses to define the specific implementation of some or all of those methods.

Applying Method Overriding

- The following program creates a superclass called **Figure** that stores the dimensions of various two-dimensional objects.
- It also defines a method called **area( )** that computes the area of an object.
- The program derives two subclasses from **Figure**. The first is **Rectangle** and the second is **Triangle**.
- Each of these subclasses overrides **area( )** so that it returns the area of a rectangle and a triangle, respectively.

```
// Using run-time polymorphism.
class Figure
 {
double dim1;
double dim2;
Figure(double a, double b) {
dim1 = a;
dim2 = b;
}
double area() {
System.out.println
        ("Area for Figure is undefined.");
return 0;
}
}
class Rectangle extends Figure {
Rectangle(double a, double b) {
super(a, b);
}
// override area for rectangle
double area() {
System.out.println("Inside Area for
Rectangle.");
return dim1 * dim2;
}
}
class Triangle extends Figure {
Triangle(double a, double b) {
super(a, b);
}
// override area for right triangle
double area() {
System.out.println
            ("Inside Area for Triangle.");
return dim1 * dim2 / 2;
}
}
```

```
class FindAreas {
public static void main(String args[]) {
Figure f = new Figure(10, 10);    //super class object
Rectangle r = new Rectangle(9, 5);
Triangle t = new Triangle(10, 8);
Figure figref;
figref = r;
System.out.println("Area is " + figref.area());
figref = t;
System.out.println("Area is " + figref.area());
figref = f;
System.out.println("Area is " + figref.area());
}
}
```

Output:

Inside Area for Rectangle.
Area is 45
Inside Area for Triangle.
Area is 40
Area for Figure is undefined.
Area is 0